

- ✓ TTL strategy (15–30 mins)
- ✓ Explicit invalidation on role update

Impact:

- Reduced DB calls for auth by large margin
- Authorization became $O(1)$ lookup

③ APPLICATION LAYER OPTIMIZATION

- ✓ Removed unnecessary object mapping
 - Avoided deep entity graph serialization
- ✓ Used async processing for:
 - Email notifications
 - Report generation
 - Background tasks
- ✓ Used connection pooling (HikariCP tuning)
 - Optimized max pool size
 - Monitored slow queries
- ✓ Avoided excessive logging in production
 - Reduced I/O overhead

④ AUTHORIZATION OPTIMIZATION

Problem:

Heavy permission checks at service level.

Solution:

- ✓ Moved to AOP-based centralized permission check
- ✓ Avoided duplicate authorization logic in services
- ✓ Used Redis-based permission resolution

Impact:

- Cleaner code
- Faster permission validation
- Reduced redundant checks

⑤ INFRASTRUCTURE OPTIMIZATION

- ✓ Enabled API Gateway level rate limiting

- ✓ Enabled compression (GZIP)
- ✓ Proper thread pool configuration
- ✓ Monitored via logs & metrics
- ✓ Tuned Kubernetes resource limits

6 RESPONSE OPTIMIZATION

- ✓ Used pagination everywhere
- ✓ Avoided returning large nested JSON
- ✓ Used DTO instead of exposing entities
- ✓ Applied selective serialization

REAL PRODUCTION EXAMPLE

In reporting API:

Before:

- Multiple joins
- Full entity load
- No caching
- High response time

After:

- Optimized native query
- Added index
- Added pagination
- Moved heavy calculation async
- Cached static metadata

Result:

- ✓ API response time reduced
- ✓ DB load reduced
- ✓ No timeout in peak usage

SHORT INTERVIEW VERSION

"I optimized APIs at multiple layers — database indexing, query tuning, fixing N+1 issues, Redis caching for authorization, DTO projections to reduce payload size, async processing for heavy tasks, and infrastructure tuning. This significantly reduced response time and DB load while improving scalability."

5. Which endpoint/API you created?

DEPLOYED ROLE & PERMISSION MANAGEMENT APIs

We implemented dedicated APIs to manage dynamic roles and permissions in the system. These APIs are not simple CRUD endpoints — they include internal validation, mapping logic, cache handling, and security enforcement.

1 API :: getRoleList

Endpoint:

GET /api/roles

Purpose:

Fetch all available roles (System + Custom).

Internal Logic:

- Fetch roles from database
- Identify system roles (non-editable flag)
- Mark custom roles (editable)
- Apply organization-level filtering if applicable
- Return structured role response

Response Example:

```
[
  {
    "roleId": 1,
    "roleName": "ADMIN",
    "type": "SYSTEM",
    "editable": false
  },
  {
    "roleId": 7,
    "roleName": "CUSTOM_AUDITOR",
    "type": "CUSTOM",
    "editable": true
  }
]
```

Additional Handling:

- Authorization check before returning roles
- Redis caching (optional for optimization)

② API :: getPermissionListForRole

Triggered When:

User clicks on a specific role in UI.

Endpoint:

GET /api/roles/{roleId}/permissions

Purpose:

Load complete permission matrix for the selected role.

Internal Logic:

- Validate role existence
- Fetch role-permission mappings
- Group permissions by module (Survey, Audit, User Mgmt, etc.)
- Mark selected vs unselected permissions
- Handle system role restrictions (view-only if system role)

Response Example:

```
{
  "roleId": 7,
  "roleName": "CUSTOM_AUDITOR",
  "permissions": [
    {
      "module": "SURVEY",
      "permissionCode": "SURVEY_CREATE",
      "assigned": true
    },
    {
      "module": "SURVEY",
      "permissionCode": "SURVEY_DELETE",
      "assigned": false
    }
  ]
}
```

Security Handling:

- Permission to view role configuration required
- Dynamic role-permission resolution via Redis → DB fallback

③ API :: updateRolesToPermission

Triggered When:

User edits permissions and clicks SAVE.

Endpoint:

PUT /api/roles/{roleId}/permissions

Request Body:

```
{
  "permissionIds": [101, 102, 205, 309]
}
```

Purpose:

Update role-to-permission mapping.

Internal Execution Flow:

Step 1:

- Validate role exists
- Check if role is editable
- Reject update if system role

Step 2:

- Fetch existing permissions
- Calculate:
 - Permissions to add
 - Permissions to remove

Step 3:

- Perform batch insert/delete in mapping table
- Maintain transactional integrity

Step 4:

- Invalidate Redis cache:
 - ROLE_PERMISSIONS:{roleId}
 - USER_ROLES cache for impacted users

Step 5:

- Optional audit **logging** of changes

Response:

```
{
  "status": "SUCCESS",
  "message": "Role permissions updated successfully"
}
```



Why This Design Matters

-
- ✓ Clear separation between role and permission management
 - ✓ System roles protected (non-editable)
 - ✓ Custom roles flexible and configurable
 - ✓ Redis cache keeps authorization fast
 - ✓ Cache invalidation ensures immediate reflection
 - ✓ Transaction-safe updates
 - ✓ Fully dynamic RBAC

FLOW SUMMARY

UI → getRoleList
→ Click Role → getPermissionListForRole
→ Edit → updateRolesToPermission
→ Cache invalidation
→ Dynamic authorization reflects immediately

=====

These APIs collectively power the dynamic RBAC system
deployed in production.

=====

6. What was the DB schemas you worked on?

**I primarily worked on RBAC (Role-Based Access Control) schemas
and core business schemas for an enterprise audit application.**

A) RBAC SCHEMAS (Most Critical Area)

1) users

-
- id (UUID, Primary Key)
 - username (Unique)
 - email (Unique)
 - password_hash
 - status
 - created_at
 - updated_at

Note:

UUID used because:

- Distributed microservices environment

- Avoid sequential ID exposure
- Better merge compatibility across services

2) roles

-
- id (UUID, Primary Key)
 - role_name (Unique)
 - description
 - role_type (GLOBAL / ASSESSMENT)
 - created_at
 - updated_at

3) permissions

-
- id (UUID, Primary Key)
 - permission_key (Unique)
 - module_name
 - action_type (READ / WRITE / DELETE / APPROVE)
 - created_at
 - updated_at

4) user_roles (Mapping Table)

-
- id (UUID, Primary Key)
 - user_id (UUID, FK → users.id)
 - role_id (UUID, FK → roles.id)
 - assessment_id (UUID, Nullable, for context-based role override)
 - assigned_at
 - assigned_by

5) role_permissions (Mapping Table)

-
- id (UUID, Primary Key)
 - role_id (UUID, FK → roles.id)
 - permission_id (UUID, FK → permissions.id)
 - created_at

Why UUID here?

- Security-sensitive module
- Prevent predictable ID enumeration
- Used across microservices
- Compatible with Redis caching keys

B) SURVEY / RESPONSE SCHEMA

6) survey

-
- id (UUID, Primary Key)
 - title
 - description
 - created_by (UUID)
 - created_at
 - updated_at

7) question

-
- id (UUID, Primary Key)
 - question_text
 - question_type
 - is_mandatory
 - created_at

8) survey_assignment

-
- id (UUID, Primary Key)
 - survey_id (UUID, FK)
 - user_id (UUID, FK)
 - assigned_at
 - due_date
 - status

9) survey_response

-
- id (UUID, Primary Key)
 - user_id (UUID, FK)
 - survey_id (UUID, FK)
 - question_id (UUID, FK)
 - response
 - submitted_at

Composite Unique Constraint:

UNIQUE (user_id, survey_id, question_id)

Purpose:

Prevent duplicate entries under concurrent API calls.

C) GENERIC CORE BUSINESS SCHEMAS

10) audit

- id (UUID, Primary Key)
- audit_name
- audit_type
- status
- start_date
- end_date
- created_by (UUID)

11) findings

- id (UUID, Primary Key)
- audit_id (UUID, FK)
- severity
- description
- status
- owner_id (UUID)
- created_at

12) documents

- id (UUID, Primary Key)
- reference_id (UUID)
- reference_type
- s3_path
- uploaded_by (UUID)
- uploaded_at

13) notifications

- id (UUID, Primary Key)
 - user_id (UUID)
 - message
 - is_read
 - created_at
-

Why UUID Instead of Auto-Increment?

- Microservices architecture
- Avoid collision during data migration
- Prevent ID guessing
- Easier replication & sharding
- Safe for distributed systems

Summary (Interview Ready)

I worked extensively on:

- 1) RBAC schemas (users, roles, permissions, mappings)
- 2) Context-based authorization schemas
- 3) Survey & response management schemas
- 4) Audit core business schemas
- 5) Document & notification schemas

Most tables used UUID as primary key
to support distributed architecture, security, and scalability.

7. How implementing Redis & Kafka improved response time?

8. How is performance testing done in your project?

9. How do you know response code improved by x second?

10. SDLC of your project

11. Tell when your responsibility over feature you delivered

12. How do you know when Redis cache should flush out the data & where do you write the logic?

13. do you know multithreading and how you implemented in your project

14. not reproducible issue?

Yes — this duplicate survey response issue was not consistently reproducible.

How We Tried to Recreate It:

- 1) Sent multiple rapid requests from frontend.
- 2) Used Fiddler to fire parallel API requests.
- 3) Enabled Chrome DevTools network throttling (3G mode).
- 4) Tested with concurrent sessions.

Still not reproducible consistently.

Observation:

- It was a race condition.

- Timing dependent.
- Occurred only under specific production latency scenarios.
- Non-deterministic in local/dev environments.

Resolution Approach:

Instead of chasing reproduction,
we focused on structural prevention.

Upgraded schema and enforced:

UNIQUE (user_id, survey_id, question_id)

Now:

- Even if any client sends duplicate concurrent requests,
- Even if retries happen due to network,
- Even if new frontend is integrated,

Database prevents duplicate entries.

Key Learning:

- Some concurrency bugs are rare and timing-sensitive.
- Do not rely only on application-level fixes.
- Enforce critical business rules at database level.
- Fix the architecture, not just the symptom.

16. Why do we need integer pk if we have uuid?

=> UUID ensures global uniqueness and prevents ID enumeration. However, BIGINT primary keys provide better indexing, faster joins, lower storage cost, and improved write performance due to sequential inserts. So we use BIGINT internally for database efficiency and UUID externally for security and distributed system compatibility.

Authentication

Authentication in our project is implemented using JWT (JSON Web Token) and follows a stateless authentication model.

The application is stateless, and authentication is handled at the API Gateway level. Spring Security is used to validate and authenticate requests using a chain of security filters.

When a user logs in:

- If no JWT is present, the API Gateway validates the user credentials (user ID and password) against the database.
- Once the credentials are successfully verified, the system generates a JWT.

The generated JWT contains:

- User ID
- User roles
- Token metadata (expiry, issuer, etc.)

Two types of tokens are issued:

1. Access Token
2. Refresh Token

The Access Token:

- Is short-lived
- Is stored in the frontend Redux store
- Is sent with every API request in the Authorization header
- Is used to authenticate and authorize the user for each request

The Refresh Token:

- Is long-lived
- Is stored securely in browser cookies
- Is used only when the access token expires or becomes invalid

For every incoming request:

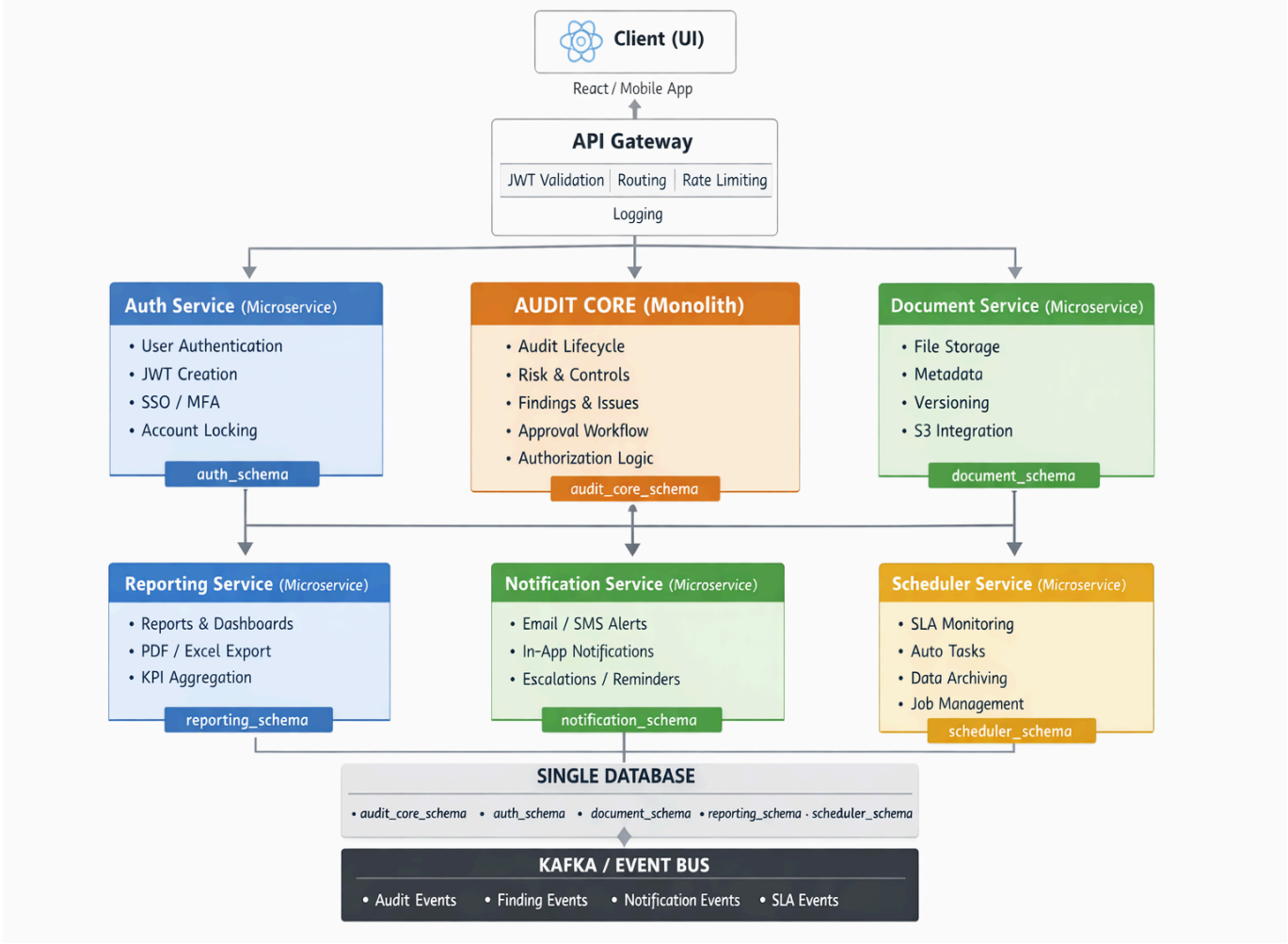
- The API Gateway validates the access token using Spring Security filters
- If the token is missing, expired, the request is rejected

If the access token is expired:

- The refresh token is sent to generate a new access token
- If the refresh token is also invalid, the user is logged out and access is denied

Overall, authentication is enforced through a combination of JWT-based stateless security and Spring Security filter chains.

Diagram



API created

Authorisation

Authorisation

Previous rbac

```
=====
PREVIOUS AUTHORIZATION SETUP
(Static / Hard-coded Roles)
=====
```

1) PREDEFINED ROLES IN THE SYSTEM

The application had predefined roles already configured in the backend code.
These roles were fixed and could NOT be modified by the admin.

Example predefined roles:

ADMIN
AUDITOR
REVIEWER
OBSERVER

Internally Spring Security stores roles with prefix ROLE_ :

ROLE_ADMIN
ROLE_AUDITOR
ROLE_REVIEWER
ROLE_OBSERVER

2) ADMIN CREATES USER AND ASSIGNS ROLE

Admin could create a new user and assign one of the predefined roles.

Example:

User	Assigned Role
Tejas	ADMIN
Rahul	AUDITOR
Priya	REVIEWER

Example DB record:

username : rahul
role : ROLE_AUDITOR

3) PROTECTING APIs USING @PreAuthorize

To restrict access to APIs we used Spring Security annotation @PreAuthorize.

Example:

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteAudit(Long id){
    auditRepository.deleteById(id);
}
```

Meaning:

Only users with ADMIN role can execute this method.

4) HOW @PreAuthorize CHECKS ROLE

When request comes:

```
Client Request
↓
JWT Token Validated
↓
Roles extracted from token
↓
Authentication object created
↓
Stored inside SecurityContext
```

Example SecurityContext:

User : rahul
Authorities : [ROLE_AUDITOR]

Now Spring evaluates:

```
@PreAuthorize("hasRole('ADMIN')")
```

Spring internally converts this to:

ROLE_ADMIN

Then it checks:

Does ROLE_ADMIN exist in authentication authorities?

5) FINAL AUTHORIZATION DECISION

CASE 1 : ROLE PRESENT

Authorities : [ROLE_ADMIN]

Required : ROLE_ADMIN

Result → Access Granted

Method executes

CASE 2 : ROLE NOT PRESENT

Authorities : [ROLE_AUDITOR]

Required : ROLE_ADMIN

Result → AccessDeniedException

HTTP 403 Forbidden

Method execution blocked

SHORT SUMMARY

- Roles were predefined in the system
- Admin could only assign roles to users
- APIs were protected using @PreAuthorize
- Spring fetched roles from SecurityContext
- If role matched → method executed
- If role missing → access denied

RBAC AUTHORIZATION FLOW – PROJECT NOTES

(Multi-Tenant + Assessment Based Access)

SYSTEM IDENTIFIERS

tenantId → identifies client organization

userId (UUID) → identifies user

assessmentId → identifies audit/assessment context

Authorization decision is based on:

tenantId + userId + assessmentId

=====

=====

A) ADMIN OPERATIONS

1. ADMIN CREATES USER AND ASSIGNS ROLE

Admin creates a user from the UI and assigns role per assessment.

Example UI action:

User Email: tejas@gmail.com

Tenant: ABC Bank

Assessment: A101

Role: AUDITOR

DATABASE TABLES USED

TABLE: users

user_id (UUID)
tenant_id
email
name
status

Example:

user_id = U123
tenant_id = T100
email = tejas@gmail.com

TABLE: assessments

assessment_id
tenant_id
assessment_name

Example:

assessment_id = A101

TABLE: user_assessment_roles

Stores role of a user for a specific assessment.

id
tenant_id
user_id
assessment_id
role_id
assigned_at

Example record:

tenant_id T100
user_id U123
assessment_id A101
role_id AUDITOR

Meaning:

User U123 is AUDITOR in assessment A101

2. ROLE → PERMISSION MAPPING

Roles and permissions are stored separately.

TABLE: roles

role_id
role_name

Example:

ADMIN
AUDITOR
REVIEWER

TABLE: permissions

permission_id
permission_key

Example:

AUDIT_CREATE
AUDIT_EDIT
WORKPAPER_UPLOAD
REPORT_EXPORT

TABLE: role_permissions

role_id
permission_id

Example records:

AUDITOR → AUDIT_CREATE
AUDITOR → AUDIT_EDIT
AUDITOR → WORKPAPER_UPLOAD

REVIEWER → AUDIT_REVIEW
REVIEWER → WORKPAPER_APPROVE

ADMIN → all permissions

=====

B) USER REQUEST FLOW

1. HOW REQUEST URL LOOKS

Example request:

POST /api/v1/assessments/A101/workpapers

Headers:

Authorization: Bearer JWT_TOKEN

From URL we extract:

assessmentId = A101

2. JWT PAYLOAD CONTENT

JWT contains identity information only.

Example payload:

```
{
  "tenantId": "T100",
  "userId": "U123",
  "email": "tejas@gmail.com", or we can send uuid
  "exp": 1712345678
}
```

Important fields used:

tenantId
userId

These values are extracted during authentication.

3. CUSTOM AUTHORIZATION ANNOTATION

APIs are secured using a custom annotation.

Example:

```
@RequirePermission("WORKPAPER_UPLOAD")
```

API example:

```
@RequirePermission("WORKPAPER_UPLOAD")
@PostMapping("/assessments/{assessmentId}/workpapers")
public ResponseEntity uploadWorkpaper() {
    ...
}
```

Meaning:

Only users having permission WORKPAPER_UPLOAD
for that assessment can execute this API.

4. AOP AUTHORIZATION ASPECT

The aspect intercepts the API and checks permission.

```
@Aspect
@Component
public class AuthorizationAspect {

    @Autowired
    private RedisService redisService;

    @Autowired
    private RoleService roleService;
```

```

@Before("@annotation(requirePermission)")
public void checkPermission(RequirePermission requirePermission) {

    String requiredPermission = requirePermission.value();

    Authentication auth =
        SecurityContextHolder.getContext().getAuthentication();

    String userId = auth.getName();

    HttpServletRequest request =
        ((ServletRequestAttributes) RequestContextHolder
            .getRequestAttributes()).getRequest();

    String tenantId = request.getHeader("tenantId");
    String assessmentId = request.getParameter("assessmentId");

    String role = redisService.getUserRole(tenantId, userId, assessmentId);

    Set<String> permissions = redisService.getPermissionsForRole(role);

    if(!permissions.contains(requiredPermission)){
        throw new AccessDeniedException("Permission denied");
    }
}
}

```

5. REDIS CONFIGURATION AND METHODS

Redis is used as a caching layer for RBAC.

Main Redis Keys:

User role per assessment:

role:user:{tenantId}:{userId}:{assessmentId}

Example:

role:user:T100:U123:A101 → AUDITOR

Role permissions: